Questions & Answers

1. Technical Challenges Describe the greatest challenge(s) you encountered in translating the template from CloudFormation to Terraform. (1-2 paragraphs)

- Having not as much exposure or understanding of Terraform as I did CloudFormation, when mentioning the cf2tf tool, I was hoping that working on polishing my CloudFormation template earlier then using the package would give me a polished terraform template. However, when converting it over, it was clear that the package did not do as nearly as I intended it to, and was quickly hit with a whole host of issues and bugs that seemingly took a while to debug. Given the complexity in both templates, I had to thoroughly test both templates with a variety of tests to make sure that I was hitting everything that I needed to.
- While working in terraform itself, understanding how resource attributes related to and depended on another was difficult to grasp, especially when using embedded variables in order to substitute for certain variables. This meant that understanding and translating how resources relate to one another, and just the overall syntax of the template in Terraform was for me, a little bit more complex to spend term learning what was going on.

2. Access Permissions What element (specify file and line #) grants the SNS subscription permission to send messages to your API? Locate and explain your answer.

```
SnsSubscription:
Type: AWS::SNS::Subscription
DependsOn: ElasticIpAssociation
Properties:
TopicArn: !Ref SnsTopic
Protocol: http # given
Endpoint: !Sub http://${ElasticIp}:8000/notify # endpoint e.g. http://<Elastic-IP>:8000/notify
```

- build.yaml // Lines 218 – 224
- In this section of the template, an SNS subscription is created which connects the SNS topic to the FastAPI app. More specifically, Line 223 indicates that we will send messages through HTTP requests. Line 224, 'Endpoint', specifies the URL of the FASTAPI endpoint. Thus, when SNS receives a message, in this case S3 uploading a file, an HTTP request is sent to the EC2 /notify. This subscription allows us to have SNS deliver messages to our specified endpoint.
- Thus, the element that lets SNS have the ability to send messages is the Endpoint in line 224, which connects SNS --> API.

3. Event flow and reliability: Trace the path of a single CSV file from the moment it is uploaded to raw/ in S3 until the FastAPI app processes it. What happens if the EC2 instance is down or the /notify endpoint returns an error? How does SNS behave (e.g., retries, dead-letter behavior), and what would you change if this needed to be production-grade?

- Starting from the beginning, where a single CSV file is uploaded to raw/ in S3, an ObjectCreated event comes out because we specified for csv files. This is pushed to SNS topic, which an HTTP notification is sent to the FASAPI endpoint at /notify.
- Inside the /notify, the SNS subscription is confirmed, with the event Json embedded, where it is parsed, checks for the .csv file type, and then processes the file based on the bucket and object key. Here, the processing file downloads the CSV, uses the current baseline, updates, checks for any anomalies, writes a processed CSV files, and saves and uploads the log and baseline files. Importantly, here we do not trigger any notifications since none of these files are filtered for raw csv files.
- If the EC2 instance is down, the SNS tries but fails to send the HTTP notification to /notify, which retries based on AWS policies, and by default has delays between reattempts. If there is a different type of error, such as permanent failure, but we configure it to be retriable from SNS's point of view. It does not saved failed delivers based on dead letter behavior.
- Given all these factors, we would have to adjust a couple things to make it production grade. AWS policies indicate that we would be recommended to avoid HTTP endpoints for SNS delivery in a production setting. In addition, to avoid the issues with dead letter behavior, you might implement a dead-letter queue so you don't lose messages if something goes wrong and you just keep on retrying deliveries. Through additional resources, adding a buffer into the mix, such as SQS queues, in order to contract any issues if something breaks, or messages or topics cannot be processed.

4. IAM and least privilege: The IAM policy for the EC2 instance grants full access to one S3 bucket. List the specific S3 operations the application actually performs (e.g., GetObject, PutObject, ListBucket). Could you replace the "full access" policy with a minimal set of permissions that still allows the app to work? What would that policy look like?
- Operations Actually Performed: s3:ListBucket, s3:GetObject, s3: PutObject

- We don't need actions like DeleteObject, or CreateBucket, which is too broad and outside the scope needed. As a result, we can follow the principle of least privilege, by only allowing bucket listing upon the bucket itself and Get/Put on the objects in the bucket itself, specifying these outright and avoiding any extra unnecessary access.

5. Architecture and scaling: This solution uses batch-file events (S3 + SNS) to drive processing, with a rolling statistical baseline in memory and in S3. How would the design change if you needed to handle 100x more CSV files per hour, or if multiple EC2 instances were processing files from the same bucket? Address consistency of the shared baseline.json, concurrent processing, and any tradeoffs.

https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/BestPractices_ImplementingVersionControl.html

- Given the need to handle a significantly larger load, we could revisit the aforementioned queue system to provide a buffer and thus develop some parallel processing. However, since we could only update the base line essentially one at a time, adding multiple instances means the baseline.json file that we were look at would be editing and updating from multiple sources at once. To handle the increase in number of files, the queue will make sure that even if something breaks, you can figure it out later even with any issues going on. Rather than having a single json object, we can look at AWS documentation, which uses DynamoDB and has conditional writes and optimistic looking . This means that in all the updating, it checks for the current version or if its old, and if it is an old file, then we fail the process instead of overwriting whatever update came form a different time. This allows for safe concurrent processing.
- Here, we are looking to balance the trade off between simplicity and scalability. Our design in this project only had to use 1 EC2 instance and JSON baseline file, which simple, and works relatively effectively, but only works at a small scale. Other architectures explored in this write up, such as with DynamoDB, allows us to expand vertically and horizontally, using multiple instances, and handles any concurrency and processing issues that come from the increased complexity.